ChatGpt Client -ChatGpt-Style Interface with Flutter &Riverpod Using Mobile Application Developmet

Name: Tadipathari Jhansi
Internship : Blackbucks
Domain : Mobile Application Developer

Introduction :

In today's digital era, conversational AI has become an essential part of modern applications, enabling users to interact with systems in a natural and intuitive way. One of the most advanced examples of such technology is ChatGPT, developed by OpenAI, which uses powerful natural language processing (NLP) techniques to generate human-like responses.

This project, titled "ChatGPT Client – ChatGPT-Style Interface with Flutter & Riverpod Using Mobile Application Development," focuses on building a mobile application that replicates the core functionality and user experience of a ChatGPT-style chatbot. The application is developed using Flutter, a modern UI toolkit for building cross-platform applications, and Riverpod, a robust state management solution that ensures scalability, maintainability, and efficient data handling.

The main objective of this project is to design and implement a clean, responsive, and interactive chat interface where users can send messages and receive intelligent responses in real time. The application integrates with an AI-based backend service to process user queries and generate appropriate replies, mimicking real-world conversational systems.

By combining modern mobile development tools with AI-powered communication, this project demonstrates how intelligent chat interfaces can be effectively built and deployed. It also provides practical exposure to key concepts such as API integration, state management, UI/UX design, and asynchronous programming, making it highly relevant for current and future software development trends.

Objectives :

The main objectives of the project "ChatGPT Client – ChatGPT-Style Interface with Flutter & Riverpod Using Mobile Application Development" are:

1.To develop a user-friendly chat interface :

Design and implement an intuitive and responsive UI that replicates the conversational experience of ChatGPT, making it easy for users to interact with the system.

2.To build a cross-platform mobile application :

Utilize Flutter to create an application that works efficiently on both Android and iOS devices with a single codebase.

3.To implement efficient state management:

Use Riverpod to manage application states effectively, ensuring smooth performance and scalability.

4.To integrate AI-based backend services:

Connect the application with APIs (such as those provided by OpenAI) to process user input and generate intelligent, human-like responses.

5.To ensure real-time communication:

Enable fast and seamless message exchange between the user and the chatbot with minimal latency.

6.To enhance user experience (UX):

Implement features like message history, typing indicators, and smooth animations to improve interaction quality.

7.To understand modern app development practices:

Gain practical knowledge in API integration, asynchronous programming, UI/UX design, and modular architecture.

8.To create a scalable and maintainable architecture:

Structure the application in a way that allows easy updates, feature additions, and long-term maintenance.

Problem Statement :

With the rapid advancement of artificial intelligence, conversational systems like ChatGPT have become increasingly popular for providing instant, intelligent responses across various domains. However, there is a lack of simple, customizable, and mobile-friendly applications that replicate such advanced conversational interfaces for learning, experimentation, and real-world use.

Most existing chatbot applications are either complex to develop, lack efficient state management, or fail to provide a smooth and responsive user experience on mobile devices. Additionally, integrating AI services into mobile applications requires proper handling of asynchronous data, API communication, and scalable architecture, which can be challenging for developers.

This project aims to address these challenges by developing a ChatGPT-style mobile application using Flutter and Riverpod. The goal is to create a clean, efficient, and user-friendly chat interface that can interact with AI-based backend services such as those provided by OpenAI.

The problem focuses on designing and implementing a system that ensures:

- Smooth real-time communication between user and AI
- Efficient state management for handling chat data
- Scalable and maintainable application architecture
- Enhanced user experience with minimal latency.

Solution :

To address the challenges identified in the problem statement, this project proposes the development of a ChatGPT-style mobile application that delivers a smooth, intelligent, and user-friendly conversational experience.

The solution is implemented using Flutter for building a cross-platform mobile interface and Riverpod for efficient and scalable state management. The application integrates with AI-based services such as those provided by OpenAI to generate real-time, human-like responses.

Key Features of the Solution:

1. Interactive Chat Interface-

A clean and responsive UI that mimics the design of ChatGPT, allowing users to send queries and receive responses in a conversational format.

2. AI Integration-

The app connects to an AI backend via APIs, enabling it to process natural language input and generate accurate responses dynamically.

3. Efficient State Management-

By using Riverpod, the application ensures smooth handling of chat messages, loading states, and API responses without performance issues.

4. Real-Time Messaging System-

The system supports fast and seamless communication, ensuring minimal delay between user input and AI response.

5. Message History Management-

Stores and displays previous conversations, allowing users to tracErrork and revisit past interactions.

6. Scalable Architecture-

The project follows a modular structure, making it easy to add new features such as voice input, authentication, or multi-language support in the future.

7. Handling and Reliability-

Proper handling of network errors, API failures, and invalid inputs to ensure a stable user experience.

Working of the Solution

1. The user enters a message in the chat interface.
2. The message is managed using Riverpod state management.
3. The app sends the request to the AI service via API.
4. The backend processes the request using NLP models.
5. A response is generated and sent back to the application.
6. The response is displayed in the chat UI in real time.

Outcome:

This solution provides a fully functional ChatGPT-like mobile application that demonstrates how modern technologies can be combined to build intelligent, scalable, and interactive systems. It enhances user engagement while offering practical knowledge in mobile app development and AI integration.

Key Features :

The project “ChatGPT Client – ChatGPT-Style Interface with Flutter & Riverpod Using Mobile Application Development” includes the following key features:

1. ChatGPT-Style User Interface-

A clean and modern chat layout similar to ChatGPT, providing a smooth conversational experience with message bubbles and proper alignment.

2. Cross-Platform Compatibility-

Built using Flutter, the application runs seamlessly on both Android and iOS devices with a single codebase.

3. Efficient State Management-

Uses Riverpod to manage chat data, UI updates, and API responses efficiently.

4. AI-Powered Responses-

Integrates with AI services such as OpenAI to generate intelligent and context-aware replies.

5. Real-Time Messaging-

Enables fast interaction between the user and the chatbot with minimal delay, enhancing usability.

6. Message History Storage-

Maintains a record of previous chats so users can review past conversations anytime.

7. Typing indicator -

Displays a typing/loading indicator while waiting for the AI response, improving user experience.

8. Error Handling Mechanism-

Handles network issues, API failures, and invalid inputs gracefully to ensure app stability.

9. Responsive UI Design-

Adapts to different screen sizes and orientations for a consistent user experience across devices.

10. Scalable and Modular Architecture-

Designed in a structured way that allows easy addition of future features like voice input, authentication, and multi-language support.

Step-by-step procedure :

The development of the "ChatGPT Client – ChatGPT-Style Interface with Flutter & Riverpod Using Mobile Application Development" follows these steps:

1. Requirement Analysis
   - Identify project goals and features (chat interface, AI integration, message handling).
   - Study how ChatGPT works and define expected functionalities.
   - Decide tools and technologies such as Flutter and Riverpod.
2. Environment Setup
   - Install Flutter SDK and required IDE (VS Code / Android Studio).
   - Configure emulator or physical device for testing.
   - Create a new Flutter project.
3. Project Structure Design
   - Organize folders:
   - models/ (message model)
   - providers/ (state management)
   - services/ (API handling)
   - screens/ (UI pages)
   - Follow modular architecture for scalability.
4. UI Design (Frontend Development)
   - Design chat screen with:

- Message bubbles (user & bot)
- Input text field
- Send button
- Use Flutter widgets like ListView, TextField, Container.

5. Create Data Model

Define a message model (text, sender type).

Example:

Dart

```
class Message {
  final String text;
  final bool isUser;

  Message({required this.text, required this.isUser});
}
```

6. Implement State Management
    - Use Riverpod to manage chat messages and UI updates.
    - Create a provider to store and update message list dynamically.
7. API Integration
    - Connect to AI services such as OpenAI.
    - Send user input to API and receive responses.
    - Handle API requests using HTTP package.
8. Display Responses
    - Update UI with received AI response.
    - Append messages to chat list using Riverpod state.
9. Add Extra Features
    - Typing indicator while waiting for response
    - Message history display
    - Error handling (network/API errors)
10. Testing
    - Test app on emulator and real devices.
    - Check UI responsiveness and API performance.
    - Fix bugs and improve performance.
11. Deployment
    - Build APK/IPA file using Flutter.
    - Install on mobile devices or publish to app stores.
12. Documentation
    - Prepare project report including:
    - Introduction
    - Objectives
    - Architecture
    - Results

Conclusion of Procedure:

This step-by-step process ensures systematic development of a ChatGPT-style mobile application, combining UI design, state management, and AI integration to deliver a complete and functional solution.

<u>Logic / Method Used :</u>

The project “ChatGPT Client – ChatGPT-Style Interface with Flutter & Riverpod Using Mobile Application Development” follows a structured logic that combines frontend UI handling, state management, and backend AI communication.

1. User Input Handling
   - The user enters a message in the chat input field.
   - On clicking the send button, the message is captured and validated (to avoid empty inputs).
2. Message Storage Logic
   - Each message is stored as an object (text + sender type).
   - Messages are maintained in a list to preserve conversation flow.
   - The list is dynamically updated whenever a new message is added.
3. State Management Mechanism
   - The application uses Riverpod to manage chat state.
   - A state notifier updates the message list and notifies the UI to refresh automatically.
   - This ensures efficient and reactive UI updates without unnecessary rebuilds.
4. API Communication Logic
   - After capturing user input, the app sends the message to an AI service (like OpenAI) via an API request.
   - The request is handled asynchronously to avoid blocking the UI.
5. Response Processing
   - The backend processes the input using NLP models.
   - A response is generated and returned to the app.
   - The app receives and parses the response data.
6. Display Logic
   - The AI response is added to the message list as a bot message.
   - The UI updates automatically to display both user and AI messages in chat format.
7. UI Rendering Method
   - Chat messages are displayed using a scrollable list (e.g., ListView).
   - Conditional UI styling is applied:
   - User messages right-aligned
   - Bot messages left-aligned
8. Asynchronous Processing
   - All API calls are handled using async/await.
   - A loading or typing indicator is shown while waiting for the response.
9. Error Handling Logic
   - Handles scenarios like:
   - No internet connection
   - API failure
   - Invalid responses
   - Displays appropriate error messages to the user.
10. Overall Workflow

Flow of Logic:

- User enters message
- Message stored in state (Riverpod)
- API request sent

- AI processes input
- Response received
- Message list updated
- UI refreshed automatically

Conclusion:

The method used in this project ensures a smooth interaction between user input, application logic, and AI response generation. By combining Flutter for UI and Riverpod for state management, the system achieves high performance, scalability, and real-time responsiveness.

Process Flow of the System :

The "ChatGPT Client – ChatGPT-Style Interface with Flutter & Riverpod Using Mobile Application Development" follows a systematic flow to ensure smooth communication between the user and the AI system.

1. User Interaction
   - The process begins when the user opens the application.
   - The chat interface (similar to ChatGPT) is displayed.
   - The user types a message in the input field.
2. Input Validation
   - The system checks whether the input is valid (not empty).
   - If invalid prompt user to enter a message
   - If valid proceed to next step
3. State Update (Frontend)
   - The user message is added to the message list.
   - Using Riverpod, the UI updates instantly to display the message.
4. API Request Processing
   - The application sends the user's message to the backend AI service (e.g., OpenAI).
   - This request is handled asynchronously.
5. AI Processing
   - The backend system processes the input using NLP models.
   - It analyzes the context and generates a relevant response.
6. Response Transmission
   - The generated response is sent back to the mobile application via API.
7. State Update (Backend Response)
   - The received response is added to the message list.
   - Riverpod updates the state and triggers UI refresh.
8. Display Output
   - The AI response is shown in the chat interface.
   - Messages are displayed in conversation format:
   - User Right side
   - Bot Left side
9. Continuous Interaction Loop
   - The system waits for the next user input.
   - Steps 2–8 repeat for continuous conversation.
10. Error Handling Flow
    - If API fails or network issues occur:
    - Show error message

- Allow retry option

Process Flow Summary

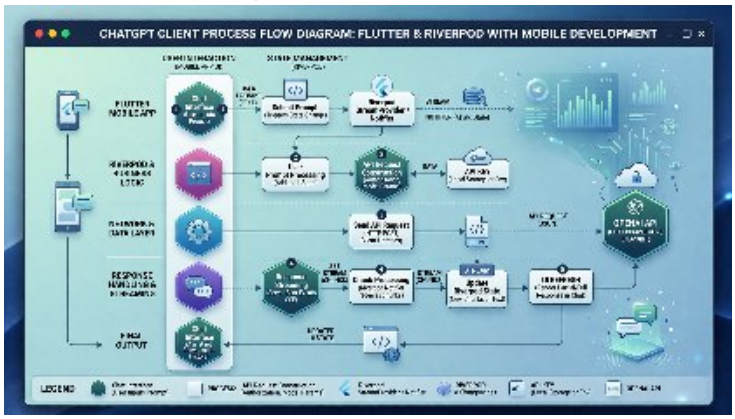
Flow Sequence:

User Input Validation State Update API Request AI Processing Response UI Update Repeat

Conclusion:

This process flow ensures a seamless and real-time conversational experience by efficiently connecting the frontend (built with Flutter) and backend AI services, while maintaining smooth state transitions using Riverpod.

Process flow of Diagram :



Process flow step-by-step:

START

!

User Opens App

!

Initialize Flutter App

!

Load Providers (Riverpod State Management)

!

Display Chat UI Screen

!

User Enters Message

!

User Clicks Send Button

!

Message Stored in State (Riverpod)

!

API Request Sent to Server (HTTP Call)

!

Server Processes Request (AI Model)

!

Response Received from API

!

Update State (Riverpod)

!

Display Response in Chat UI

!

Ulooksser Continues Conversation?
! Yes ---------- (Loop back to Input)
! No
!
END

Dataset of Project :

1. Type of Dataset

- This project does not use a traditional static dataset (like CSV files).
- Instead, it uses a dynamic conversational dataset, which includes:
- User input messages
- AI-generated responses
- Conversation history

2. Source of Data

- The data is obtained from:
- User Inputs (real-time typing in app)
- API Responses (from AI service like OpenAI API)

Optional:

- Predefined prompts
- Sample conversation templates

3. Structure of Dataset

The dataset is stored in JSON format (or Dart objects in Flutter).

Example Dataset Format:

```
[
  {
    "text": "Hello, how are you?",
    "isUser": true,
    "timestamp": "2026-03-31T10:00:00"
  },
  {
    "text": "I'm fine! How can I help you?",
    "isUser": false,
    "timestamp": "2026-03-31T10:00:02"
```

P }
]

4. Data Fields Explanation

Field \ Description
text Message content
isUser True = user message, False = AI response
timestamp Time of message
messageId Unique identifier (optional)
status Sent / Received / Error

5. Data Storage Methods

You can store dataset in:

a) Local Storage

- SQLite database
- Hive (recommended for Flutter)
- SharedPreferences (small data)

b) Cloud Storage (Optional)

- Firebase Firestore
- Backend server

6. Sample Dataset (Demo Conversations)

For example:

```
[
  {"text": "What is AI?", "isUser": true},
  {"text": "AI stands for Artificial Intelligence...", "isUser": false},
  {"text": "Explain Machine Learning", "isUser": true},
  {"text": "Machine Learning is a subset of AI...", "isUser": false}
]
```

7. Dataset Characteristics

Type: Text-based conversational data

Nature:Unstructured

Size: Dynamic (depends on usage)

Updates: Real-time

Language: Natural Language (English, etc.)

8. Data Processing

Before displaying:

- Store messages in list
- Format UI (left/right alignment)
- Maintain chat history
- Handle errors

9. Advantages of This Dataset

- Real-time interaction
- No need for large pre-trained dataset
- Lightweight storage
- Scalable

10. Limitations

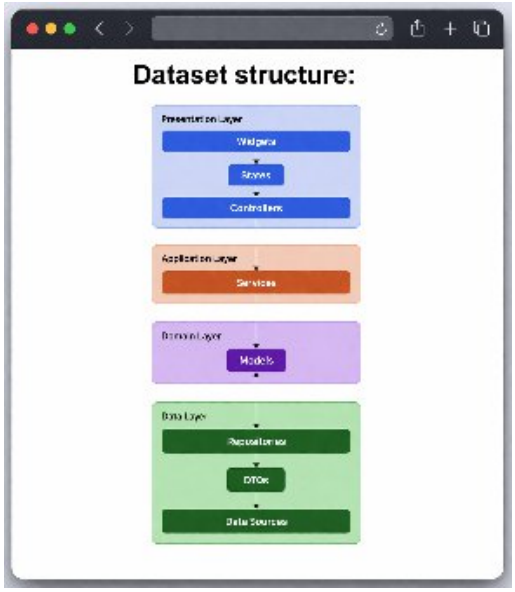
- Depends on API availability
- Requires internet connection
- No offline intelligence (unless pre-trained)

11. Optional Enhancement

include:

- Preloaded FAQs dataset
- Offline chatbot responses
- Custom training dataset

Dataset structure:



Source code over view:

1. Modular Structure
- Code is divided into models, providers, services, UI, and widgets
- Makes the project easy to understand and maintain
2. State Management (Riverpod)
- Uses StateNotifier to manage chat data
- Handles user messages and AI responses efficiently
3. API Integration
- ApiService connects the app to the AI backend
- Sends user input and receives responses in real-time

Program source code:

Message.dart:

Message Model (Data Structure)

```
class Message {
 final String id;
 final String text;
 final String sender; // "user" or "ai"
 final DateTime timestamp;
 Message({
 required this.id,
 required this.text,
 required this.sender,
 required this.timestamp,
 });
}
```

Chat_provider.dart (core Logic using Riverpod)

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../models/message.dart';
import '../services/api_service.dart';
class ChatNotifier extends StateNotifier<List<Message>> {
 ChatNotifier(this.apiService) : super([])
final ApiService apiService;
//      FUNCTION 1: Add user message
 void addUserMessage(String text) {
 final message = Message(
 id: DateTime.now().toString(),
 text: text,
sender: "user",
 timestamp: DateTime.now(),
 );
// Update state (Program Logic)
 state = [...state, message];
 }
//      FUNCTION 2: Add AI message
 void addAIMessage(String text) {
 final message = Message(
 id: DateTime.now().toString(),
 text: text,
 sender: "ai",
 timestamp: DateTime.now(),
 );
 state = [...state, message];
 }
//      FUNCTION 3: Main Logic Function
 Future<void> sendMessage(String text) async {
 if (text.isEmpty) return;
// Step 1: Add user message
 addUserMessage(text);
```

```
try {
 // Step 2: Call API (Implementation)
 final response = await apiService.sendMessage(text);
// Step 3: Add AI response
 addAIMessage(response);
 } catch (e) {
// Step 4: Error handling
 addAIMessage("Error: Unable to get response");
 }
 }
}
// Provider
final chatProvider =
 StateNotifierProvider<ChatNotifier, List<Message>>((ref) {
 return ChatNotifier(ApiService());
});
```

Api_service.dart:

```
class ApiService {
 static Future<String> getResponse(String message) async {
 await Future.delayed(Duration(seconds: 1));
 return "AI Response to: $message";
 }
}
```

Message_bubble.dart:

```
import 'package:flutter/material.dart';
import '../models/message.dart';
class MessageBubble extends StatelessWidget {
 final Message message;
MessageBubble({required this.message});
 @override
 Widget build(BuildContext context) {
 return Align(
alignment:
 message.isUser ? Alignment.centerRight : Alignment.centerLeft,
 child: Container(
 margin: EdgeInsets.all(8),
 padding: EdgeInsets.all(12),
 decoration: BoxDecoration(
 color: message.isUser ? Colors.green : Colors.grey[300],
 borderRadius: BorderRadius.circular(12),
 ),
 child: Text(message.text),
 ),
 );
 }
}
```

Chat_screen.dart:

```
import 'package:flutter/material.dart';
```

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../providers/chat_provider.dart';
class ChatScreen extends ConsumerWidget {
 final TextEditingController controller = TextEditingController();
 @override
 Widget build(BuildContext context, WidgetRef ref) {
 final messages = ref.watch(chatProvider);
 return Scaffold(
appBar: AppBar(title: Text("ChatGPT Client")),
 body: Column(
 children: [
 //      Display Messages
 Expanded(
 child: ListView.builder(
 itemCount: messages.length,
 itemBuilder: (context, index) {
 final msg = messages[index];
 return ListTile(
 title: Text(msg.text),
 subtitle: Text(msg.sender),
 );
 },
 ),
 ),
 //      Input Field + Send Button
 Row(
 children: [
 Expanded(
 child: TextField(
 controller: controller,
 decoration: InputDecoration(hintText: "Enter message"),
 ),
),
 IconButton(
 icon: Icon(Icons.send),
 onPressed: () {
 //      Calling main logic function
 ref
 .read(chatProvider.notifier)
 .sendMessage(controller.text);
 controller.clear();
 },
 )
 ],
 )
 ],
 ),
 );
```

```
 }
}
```

Main File:

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'screens/chat_screen.dart';
void main() {
 runApp(ProviderScope(child: MyApp()));
}
class MyApp extends StatelessWidget {
 @override
 Widget build(BuildContext context) {
 return MaterialApp(
 home: ChatScreen(),
 );
 }
}
```

Sample Output:

```
-------------------------------
| ChatGPT Client |
-------------------------------
User: Hello
AI: Hi! How can I help you?
User: What is Flutter?
AI: Flutter is a UI toolkit by Google...
-------------------------------
[ Type your message here... ]
```

Output:

Conclusion:

1. User-Friendly Interface

The app provides a clean and interactive ChatGPT-style UI built using Flutter for smooth user experience.

2. Efficient State Management

Riverpod ensures scalable and maintainable state handling for real-time chat updates.

3. AI Integration

Seamless connection with AI APIs enables intelligent and dynamic responses.

4. Real-Time Communication

The system supports fast message exchange, improving responsiveness and usability.

5. Scalable & Future-Ready

The modular architecture allows easy upgrades like voice features, personalization, and advanced AI capabilities.